CSC_51053 Lab: Query Optimization

Ioana Manolescu, Madhulika Mohanty

November 7, 2025

IMDB table description

- `person(id, name, gender)`
  Person is a table with information about people involved in movies. It has the people's names, as in *Garbo, Greta*, and their gender.

- `info_type(id, info)`
  Info_type comprises about 100 different categories of information which may be attached *to a movie or to a person*. For instance, one info_type is *plot*: information items of this type are attached to movies. Another info_type is *birth date*: information items of this type are attached to people.

- `link_type(id, link)`
  Link_type comprises the categories of various links between movies, such as: movie m_1 is a remake of movie m_2, m_1 is similar to m_2 etc.

- `comp_cast_type(id, kind)`
  comp_cast_type is a table of four "codes" used to characterize *the type and quality of the movie cast information available within IMDB about a specific movie*. This table is a bit unusual. First, it contains a tuple *cast* and a tuple *crew*: cast is the set of people that appear in the movie, whereas crew is the set of all people who did something for the movie (the cast plus, e.g., the make-up artists, the music composer etc.) Second, it contains a tuple *complete* and a tuple *complete+verified*, to specify whether the cast information has been verified or not. Any cast (or crew) may be complete (or, complete and verified)[1].

- `movie_type(id, kind)`
  Movie_type comprises 7 categories of movies such as: movie, TV series, video game etc.

- `role_type(id, role);`
  Role_type comprises the categories of people that are involved in a movie: actors, producers, composers, costume designers etc.

- `aka_name(id, person_id, name)`
  Aka_name is a table of alternate names for people. This table has a foreign key on person.

- `movie(id, title, kind_id, production_year, episode_of_id, season_nr, episode_nr, series_years)`
  Movie table is the main table representing movies. It has the title, production year, episode number etc. Each movie is categorized in one of these types (movie has a foreign key referencing movie_type.id). Each episode of a serie is stored in the relation movie; episode_of_id is a foreign key into movie.

- `keyword(id, keyword)`
  Keyword is a table of keywords associated to the movies.

- `person_info(id, person_id, info_type_id, info, note)`
  Person_info stores information about people, much in the way movie_info has information about movies.

- `company(id, name, country_code)`
  Company has information about companies involved in the movies; companies have an associated company type (foreign key into company_type, see below).

- `company_type(id, kind)`
  Company_type comprises 18 roles that companies may play with respect to a movie: distributor, producer, special effects etc.

[1] In comp_cast_type, a single attribute is used to encode different things. While here we use the data "as-is", this is not, in general, good design.

- `movie_company(id, movie_id, company_id, company_type_id, note)`
  Movie_company associates movies with companies: each tuple in this table has exactly three foreign keys, one into movies, into companies and into company_type.

- `aka_title(id, movie_id, title, kind_id, production_year, episode_of_id, season_nr, episode_nr, note)`
  Aka_title is a table of alternative titles (some movies are known under more than one title). This table uses the attribute movie_id to encode which movie this is an alternative title for; movie_id is not a foreign key (this column contains some values that do not designate a movie). The foreign keys are kind_id to movie_type and episode_of_id to movie.

- `char_name(id, name)`
  Char_name comprises movie character names.

- `cast_info(id, person_id, movie_id, person_role_id, note, role_id)`
  Cast_info contains associations between a person, a role, and a movie, to signify that the person played that role in that movie. Accordingly, each tuple in cast_info has a foreign key on a person, one on a movie, and one on char_name. Further, a cast_info tuple may also have another foreign key on role_type, to say what kind of role this was (actor, producer etc.) This table also has a field note, which, when not null, gives some extra information. For instance, in a tuple specifying that x was played the role of *director* in movie m, the field *note* may be used to say: "assistant: y" to denote that y was the assistant director. (The table role_type does not have an entry for assistant director.)

- `complete_cast(id, movie_id, subject_id, status_id)`
  Complete_cast characterizes the information available in the database about the complete cast of a certain movie. It has a foreign key to the movie, and: (i) one foreign key to comp_cast_type to specify whether the information available is about the *cast* or *crew*; and (ii) a second one to specify whether the cast is considered *complete* or *complete and verified*.

- `movie_rating(id, movie_id, info_type_id, info)`
  Movie_rating is a table that comprises rating information about movies. Rating information about a movie can be of up to five different types: (i) the average rating, (ii) the number of votes expressed on the movie, (iii) whether it is in the top 250 of IMDB, (iv) whether it is in the bottom 10 of IMDB, and (v) the detailed vote distribution[2].

- `movie_info(id, movie_id, info_type_id, info, note)`
  Movie_info has information about the movies, structured as follows: each tuple in movie_info is one piece of information about one movie. Thus, movie_info has a foreign key on movie.id to identify the movie. The information comprised in a movie_info tuple is in the info attribute. To enable interpreting this information (understanding what it is about), each movie_info tuple has another foreign key into info_type (see below).

- `movie_keyword(id, movie_id, keyword_id)`
  Movie_keyword associates movie titles with keywords (two foreign keys).

- `movie_link(id, movie_id, linked_movie_id, link_type_id)`
  Movie_link is a set of links between movies. Each tuple in movie_link references the first and second movie by means of foreign keys into movie.id; the fourth attribute is the type of the link (foreign key into link_type).

[2]Each detailed vote distribution information is *a string of length 10*, where each character may be either between 0 and 9, or '.' (dot), or '='. The character at position i, $1 \leq i \leq 10$, states which % of the ratings of the movie had value i (movies can be rated from 1 to 10). Thus:

- '.' specifies that 0% of the voters gave rating i;
- 0 specifies that at most 9% of the voters gave rating i;
- 1 specifies that at least 10% and less than 20% of the voters gave rating i and so on,
- 9 specifies that at least 90% but less than 100% of the voters gave the rating i,
- '=' specifies that 100% of the voters gave the rating i.